Università degli Studi di Trento Corso di Programmazione I – E3: 145935 4° Appello – 05/07/2021 – a.a. 2020/2021	**PC N°**
COGNOME ..... **NOME** ..... **MATRICOLA**	
RITIRATO ☐ **FIRMA PER RITIRO**.....	

Esame online programmazione 1

PARTE TEORICA: 16 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Teo01 – punti 5]
[TEO] Scrivere una funzione ricorsiva ***fattoriale*** che preso in input in modo opportuno un valore **k** intero positivo ritorni il suo fattoriale; `k! = k*(k-1)*(k-2)*…*2*1`.
NOTA per convenzione `0! = 1`
Scrivere una funzione ***binomiale*** che presi in input due valori interi positivi **n** e **k** ritorni il coefficiente binomiale secondo lo schema seguente, usando la funzione ***fattoriale***

$$\binom{n}{k} = \begin{cases} \dfrac{n!}{k!(n-k)!} & \text{se } 0 \le k \le n \\ 0 & \text{se } 0 \le n < k \end{cases}$$

mostrare l'invocazione della funzione ***binomiale*** dal main passando i parametri corretti e stampare sempre nel main, in modo opportuno, il valore ritornato.
Esempi di invocazione:

$$\binom{5}{2} = \frac{5!}{2!(5-2)!} = \frac{5!}{2! \cdot 3!} = 10$$

$$\binom{3}{7} = 0 \text{ in quanto } (n=3) < (k=7)$$

[Teo02 – punti 2]
[TEO] Dopo l'istruzione i+=i;
☐ Vero ☐ Falso il valore di i è raddoppiato
☐ Vero ☐ Falso il valore di i rimane invariato
☐ Vero ☐ Falso il valore di i è dimezzato
☐ Vero ☐ Falso il valore di i è azzerato

[TEO] Il compilatore segnala un errore quando:
☐ Vero ☐ Falso una variabile è dichiarata ma mai usata
☐ Vero ☐ Falso una variabile non dichiarata viene usata
☐ Vero ☐ Falso una variabile è dichiarata, non inizializzata, e usata
☐ Vero ☐ Falso una variabile viene usata prima di essere dichiarata

[TEO] per codificare il numero 257…
☐ Vero ☐ Falso servono non più di 7 bit
☐ Vero ☐ Falso servono al massimo 8 bit
☐ Vero ☐ Falso servono almeno 9 bit

☐ Vero ☐ Falso servono non meno di 10 bit

[TEO] Una funzione ricorsiva è:
☐ Vero ☐ Falso una funzione il cui codice include una chiamata alla funzione stessa, con un parametro diverso da quello in ingresso
☐ Vero ☐ Falso una funzione il cui codice include una chiamata alla funzione stessa, con un parametro uguale a quello in ingresso
☐ Vero ☐ Falso una funzione il cui codice include una chiamata a un'altra funzione, con un parametro uguale a quello in ingresso
☐ Vero ☐ Falso una funzione il cui codice include una chiamata a un'altra funzione, con un parametro diverso da quello in ingresso

[Teo03 – punti 3]
[TEO] Definire una funzione ***condizione*** che presi in input tre valori booleani **a**, **b** e **c** ritorna un valore in linea con la tavola di verità seguente:

a	**b**	**c**	**RETURN**
F	F	F	V
F	F	V	V
F	V	F	V
F	V	V	F
V	F	F	V
V	F	V	V
V	V	F	V
V	V	V	F

mostrare l'invocazione della funzione ***condizione*** dal main passando i parametri corretti e stampare sempre nel main, in modo opportuno, il valore ritornato.

[Teo04 – punti 2]
[TEO] che cosa è l'***overflow*** e quando si verifica? Che cosa succede al contenuto di una variabile intera in caso di ***overflow***? Fare un esempio esplicativo.

[Teo05 – punti 4]
[TEO]
[a] definire che cosa è un puntatore e per cosa viene utilizzato.
[b] Perché è necessario definire puntatori per ogni tipologia di dato?
[c] Se dichiaro un array in questo modo: `int vet[10];`
cosa stampa `cout<<vet;` e perchè? E questa espressione: cout `<<&(vet[0])` cosa produce?
[d] date queste dichiarazioni:

```
double vetdbl [30];
double* pvetdbl;
int* pvetint;
```

dire se le seguenti righe di codice (assegnazioni) sono corrette o sbagliate e perchè

```
pvetdbl = vetdbl;
pvetdbl = &vetdbl;
pvetdbl = *vetdbl;
pvetint = vetdbl;
pvetint = &vetdbl;
pvetint = *vetdbl;
```

PARTE PRATICA: 16 PUNTI – PUNTEGGIO MINIMO 6 PUNTI

[Cod-punti 16] (IDEWEB1: www.programiz.com/cpp-programming/online-compiler/ IDEWEB2: http://cpp.sh/) Materiale: http://www.cplusplus.com/reference/)

[COD] Date le seguenti strutture

```
typedef enum TipoV{ AUTO, MOTO, CAMPER } TipoV;
   typedef struct TVeicolo{
   char marca[20];
  unsigned int anno;
  TipoV tipo;
  TVeicolo ()   { /* da implementare */ }
  void stampa() { /* da implementare → formato: vedi punto C */ }
 } TVeicolo;
typedef struct TRevisione {
   int head, tail;  //posizione inserimento, posizione prelevamento
   int dim;         // numero elementi array cioè della codaFIFO
   int n;           // numero elementi presenti nella coda FIFO
   TVeicolo* s;     // array dinamico circolare
   TRevisione ()         { /* da implementare */ }
   bool isEmpty()        { /* da implementare */ }
   bool isFull()         { /* da implementare */ }
   void put(TVeicolo v)  { /* da implementare */ }
   TVeicolo get()        { /* da implementare */ }
   void stampa()         { /* da implementare */ }
} TRevisione;
```

[ParteA – punti 8] Completare nelle definizioni delle strutture i metodi, costruttori indicati come "da implementare" ed aggiungere eventuali altri metodi/costruttori ritenuti necessari e/o utili. Modificare e completare inoltre il **main** inserendo il seguente codice:

```
/* dichiarare un array di 3 elementi di tipo TRevisione denominato
  revisioni. Ogni lista FIFO deve avere 10 elementi nell'array */
TVeicolo veicolo;
for(int i=0; i<5; i++) {
  nuovoVeicolo(&veicolo);
  /* in funzione del valore del campo "tipo" della variabile veicolo
   inserire nell'array revisioni il valore inizializzato
   AUTO->0 MOTO->1, CAMPER->2, per esempio se tipo=CAMPER usare:
   revisioni[2].put(veicolo);
   NOTA: deve essere sempre verificato che la coda FIFO non sia piena!
  */
}
stampaRevisioni(revisioni);     //attenzione revisioni è un array
salvaRevisioni(revisioni,AUTO); //attenzione revisioni è un array
```

[ParteB – punti 3] Implementare una funzione **nuovoVeicolo** che riceve in input per riferimento una variabile di tipo **TVeicolo v** e la inizializza con:

- campo **tipo** valore casuale tra AUTO, MOTO, CAMPER;
- campo **anno** valore casuale tra 2000 e 2021 (inclusi).
- campo **marca** letto da tastiera.

[ParteC – punti 2] Implementare una funzione **stampaRevisioni** che riceve in input un array di tipo **TRevisione rev** (array di liste FIFO) e stampa per ogni lista il contenuto utilizzando opportunamente il metodo **stampa** di **TRevisione**. Nella forma (per l'enum stampare etichetta in modo opportuno):

marca **TIPO=**tipo **ANNO=**anno

Esempio: Fiat **TIPO=**AUTO **ANNO=**2009

[ParteD – punti 3] Implementare una funzione **`salvaRevisioni`** che riceve in input un array di tipo **`TRevisione rev`** (array di liste FIFO), un valore **tipo2save** di tipo **`TipoV`** ed estrae tutti gli elementi di tutte e tre le liste FIFO (metodo GET) salvandoli su un file denominato "**revisioni.txt**", nella forma:

`marca` **ANNO=**`anno`

solamente se il tipo dell'elemento estratto coincide con **tipo2save**.